GraphRAG From Scratch: An Explicit Graph Baseline, Retrieval Diagnostics, and Update-Regime Implications for Contract Analysis

Alberto Espinosa

April 20, 2026

Abstract

Graph Retrieval-Augmented Generation (GraphRAG) is widely proposed as a remedy for the limitations of flat similarity retrieval in Retrieval-Augmented Generation (RAG), especially for multi-hop queries. However, many GraphRAG implementations obscure the causal source of improvements by coupling graph construction, traversal policy, and prompting behind high-level frameworks. This paper presents a deliberately minimal and fully explicit GraphRAG baseline implemented from scratch for contract analysis. We formalize a chunk graph $G = (V, E)$ in embedding space, specify two edge families (sequential and semantic), define a weighted traversal scoring rule that trades off query relevance and path strength, and introduce retrieval-stage diagnostics (graph coverage and path coherence) that quantify graph expansion quality independently of generation. We report empirical statistics over a benchmark of clause-oriented and multi-hop contract queries, and we qualitatively audit traces captured with Langfuse. Finally, we study a practical systems question: should a GraphRAG graph be updated by incremental node insertion, or periodically rebuilt? Using basic complex-network diagnostics (average degree, clustering coefficient, and largest connected component fraction), we show that naive incremental insertion can measurably perturb global structure relative to full rebuilds. We interpret these findings via topology and information-integration considerations: while Integrated Information Theory (IIT) offers principled integration measures, exact computation is intractable at realistic graph sizes, motivating computable proxies and periodic re-optimization as safer engineering defaults.

1 Introduction

Retrieval-Augmented Generation (RAG) combines a parametric language model with a non-parametric memory, typically a vector index queried by dense similarity, to improve factual grounding and updateability [6]. Despite its success, flat retrieval often struggles with *multi-hop* queries that require combining multiple, weakly related pieces of evidence (e.g., cross-clause risk interactions in contracts). GraphRAG addresses this by retrieving not only relevant text units, but also *relationships* that structure the evidence into paths, neighborhoods, or communities [3, 9].

This work is intentionally austere. We avoid high-level GraphRAG frameworks and graph analytics libraries, and we instead isolate a small set of algorithmic primitives: cosine similarity, thresholded edge construction, adjacency-list storage, traversal scoring, and relationship-aware prompting. The goal is a baseline in which the researcher can answer two questions with minimal confounding: (i) what changes when a graph is introduced into retrieval, and (ii) what new failure modes appear when a system transitions from "retrieve top-k" to "retrieve a subgraph".

We additionally foreground a systems question that is often under-specified in GraphRAG descriptions: *graph updates*. Many practical deployments ingest new documents continuously. If

new chunks are inserted and connected via local similarity heuristics, global structure may drift, producing shortcut edges and hub effects that alter traversability and interpretability. While Integrated Information Theory (IIT) provides a principled formalism for assessing integration, exact computation is combinatorial in network size [8, 11]. We therefore treat IIT as a motivating conceptual lens rather than a directly computable metric, and we complement it with complex-network diagnostics [7] and an explicit incremental-insertion experiment.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG models combine parametric memory with retrieved documents, marginalizing over retrieved evidence during generation [6]. Dense retrieval methods such as DPR enable scalable similarity search [5]. RAG has been surveyed extensively, including discussions of evaluation and the "lost in the middle" problem for long contexts [9].

2.2 GraphRAG

GraphRAG methods augment RAG by retrieving graph-structured elements such as nodes, paths, or subgraphs, capturing relational knowledge not reducible to semantic similarity alone [3, 9]. While many systems build explicit knowledge graphs, practical GraphRAG variants can also treat text chunks as nodes with edges derived from sequence adjacency, citations, hyperlinks, or embedding similarity.

2.3 Query-focused graph summarization

Beyond survey work, recent systems emphasize query-focused graph summarization and "local-to-global" retrieval, in which graph communities provide coarse-to-fine access to evidence [2]. Our baseline does not implement community detection, but it explicitly exposes the retrieval subgraph and forces the generator to reference relations, enabling direct auditing and ablation.

2.4 Information Integration and Computational Constraints

Integrated Information Theory (IIT) proposes measures of integration based on irreducibility of a system's cause–effect structure, quantified by Φ ("big Phi") and related quantities [8, 11]. Exact evaluation typically requires searching over partitions whose count grows super-exponentially with network size (e.g., Bell numbers), leading to severe computational barriers. Several authors propose alternative measures or approximations emphasizing computability [10]. Work on estimating integrated information using algorithmic complexity and dynamic querying further highlights both promise and practical limitations, often focusing on restricted network classes [4].

3 Methods

3.1 Data and chunking

We consider a corpus of contract documents, chunked into overlapping spans and stored in a persistent vector database. Each chunk stores metadata including its source document and sequential neighbors, enabling sequential edges.

3.2 Embedding space and similarity

We embed each chunk into $\mathbb{R}^d$ using a sentence-transformer encoder. Given embeddings v_i and v_j, cosine similarity is defined as:

$$\mathrm{sim}(i,j) = \frac{v_i \cdot v_j}{\|v_i\|\|v_j\|}. \tag{1}$$

3.3 Graph construction

We explicitly construct a graph $G = (V, E)$ where each node corresponds to a chunk:

$$V = \{(id_i,\ text_i,\ v_i,\ meta_i)\}_{i=1}^{N}. \tag{2}$$

Edges include two families:

1. **Sequential edges**: $i \leftrightarrow i{+}1$ within a source document, weight $w = 1.0$.
2. **Semantic edges**: $i \leftrightarrow j$ if $\mathrm{sim}(i,j) > \tau$, with $\tau = 0.75$.

We store G as an adjacency list to keep algorithmic behavior transparent.

3.4 Graph-guided retrieval

Retrieval proceeds in two stages:

Seed retrieval. We retrieve top-k seed nodes from the vector index using the query embedding.

Traversal expansion. Starting from seeds, we expand the graph up to depth D using a weight-prioritized traversal. Each candidate node u is assigned a score:

$$\mathrm{score}(u) = \alpha \cdot \mathrm{sim}(q,u) + \beta \cdot W(u), \tag{3}$$

where $W(u)$ is the cumulative edge weight along the best known path from any seed to u. In experiments, $\alpha = 0.75$, $\beta = 0.25$, $D = 2$, and we cap the expanded node set.

3.5 Subgraph extraction and edge pruning

We induce a subgraph over expanded nodes and prune edges whose weight is below a threshold (default 0.60), yielding a compact, query-conditioned subgraph.

3.6 Generation with explicit relationships

Unlike flat RAG concatenation, GraphRAG supplies the generator with both node texts and an explicit relationship list (edges). The prompt requires the generator to reference relationships (e.g., "Node A $\rightarrow$ Node B") to support multi-hop reasoning.

Input: Query q, vector index, graph G, seed count k, depth D
Output: Expanded node set S and pruned edge set E_S
Seeds $\leftarrow$ TopK_VectorSearch(q, k);
$S \leftarrow$ Seeds;
for $d \leftarrow 1$ **to** D **do**
  Expand candidates by following adjacency edges from S;
  Score each candidate using $\alpha \cdot \text{sim}(q, u) + \beta \cdot W(u)$;
  Add highest scoring candidates into S (bounded);
end
Induce subgraph on S and prune edges with $w < \theta$;
return S, E_S;

Algorithm 1: GraphRAG retrieval (simplified)

4 Experimental Protocol

4.1 Systems compared

We compare three pipelines over the same contract corpus:

1. **RAG**: top-k vector retrieval followed by a single synthesis call.
2. **RAU**: retrieval followed by similarity-based clustering and hierarchical synthesis (cluster summaries then global synthesis).
3. **GraphRAG**: seed retrieval, graph traversal expansion, subgraph extraction, and relationship-aware prompting.

4.2 Query set

We evaluate 25 contract queries spanning (i) standard clause probes and (ii) multi-hop risk interactions (e.g., termination and liability). The full query list is reproduced in Appendix A. For generation-stage metrics that require LLM-as-a-judge, we report results on a representative subset of 7 multi-hop and high-risk queries to manage cost and runtime while preserving interpretability.

4.3 Metrics

We report retrieval-oriented diagnostics for GraphRAG:

- **graph_coverage**: overlap between expanded nodes and a proxy "relevant set" defined as the top-K vector hits (a measurable approximation in absence of labeled ground truth).
- **path_coherence**: mean edge weight over traversal steps used during expansion.
- **subgraph size**: node and edge counts after pruning.
- **latency**: retrieval-stage timing (seed search vs graph expansion).

For generation-stage comparison (subset of queries), we additionally report:

- **faithfulness**: an LLM-as-a-judge score in $[1, 10]$ evaluating whether the answer is grounded in retrieved text.
- **information density**: preserved extracted entities per 100 generated tokens.

5 Results

5.1 Summary statistics

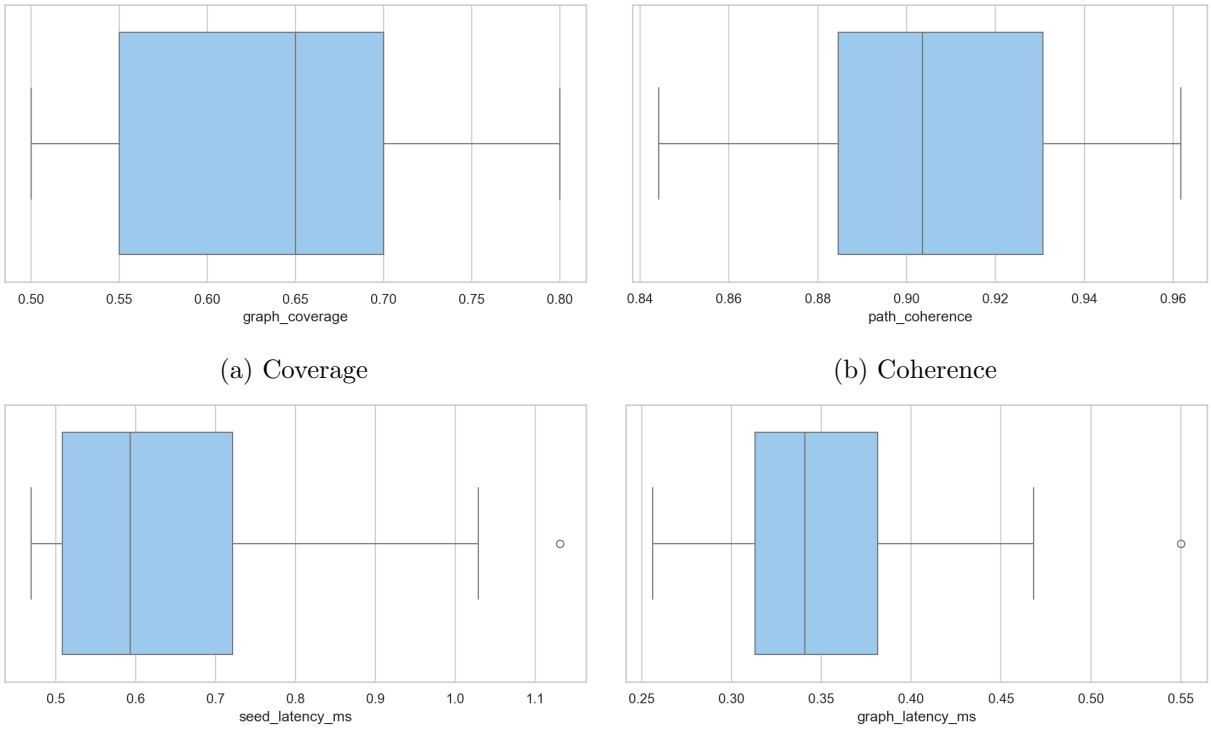
Table 1 reports aggregate statistics over the benchmark.

Table 1: GraphRAG retrieval diagnostics over 25 queries.

Metric	Mean	Std	Min	Max
Coverage	0.638	0.090	0.500	0.800
Coherence	0.904	0.032	0.844	0.962
Seed latency (ms)	0.64	0.18	0.47	1.13
Traversal latency (ms)	0.36	0.06	0.26	0.55
Expanded nodes	39.9	4.8	25.0	47.0
Expanded edges	63.8	22.6	36.0	150.0

5.2 Distributional behavior

Figure 1 summarizes the distributions of coverage, coherence, and latency components.



(a) Coverage

(b) Coherence

(c) Seed latency

(d) Traversal latency

Figure 1: GraphRAG retrieval diagnostics over 25 queries.

5.3 Subgraph size

Figure 2 illustrates the relationship between expanded node and edge counts.

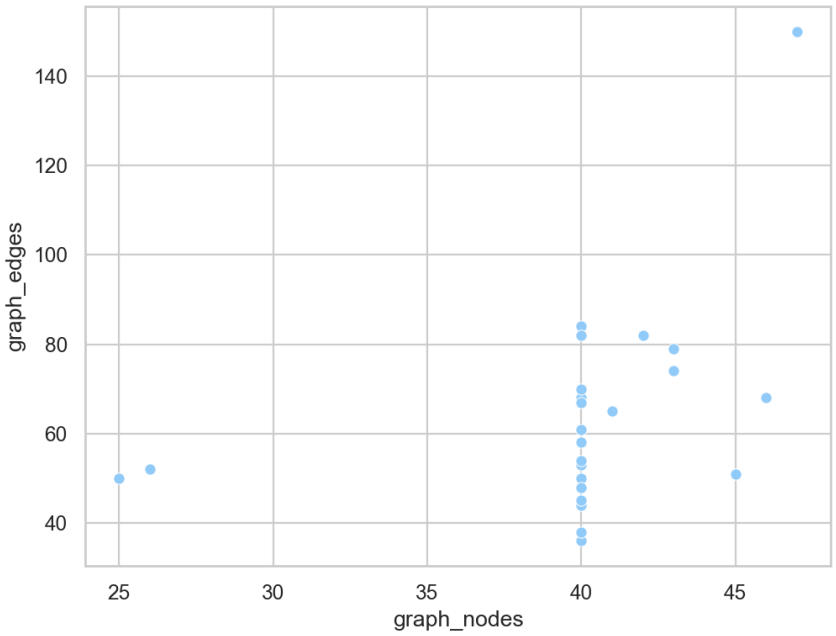


Figure 2: Expanded subgraph size (nodes vs edges).

5.4 Generation-stage metrics (subset)

Table 2 reports mean and standard deviation for generation-stage metrics over the 7-query subset. Figure 3 visualizes per-system distributions.

Table 2: Generation-stage metrics over the 7-query subset.

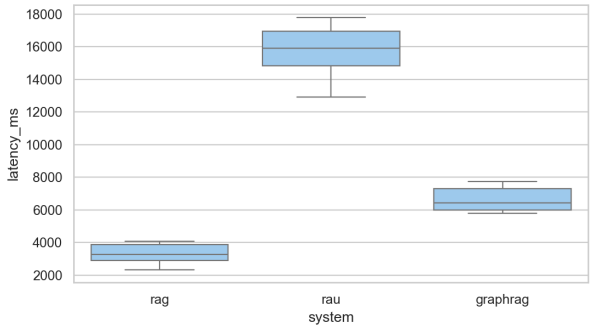
System	Latency mean	Latency std	Faith mean	Faith std	Density mean	Density std
rag	3329.8	653.7	8.14	0.38	0.123	0.164
rau	15733.0	1761.5	8.43	0.53	0.095	0.091
graphrag	6657.9	790.8	8.57	0.53	0.142	0.277

6 Langfuse Trace Interpretation (Qualitative Auditing)

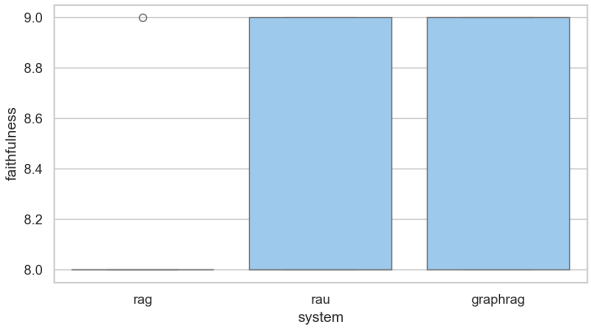
To support reproducibility and trace-based auditing, we log each pipeline run with Langfuse. In the intended setup, each path (RAG, RAU, GraphRAG) yields a trace capturing:

- the prompt payload (including graph relationships for GraphRAG),
- intermediate summarizations (cluster summaries for RAU),
- latency measurements, and
- reported scores (when enabled).

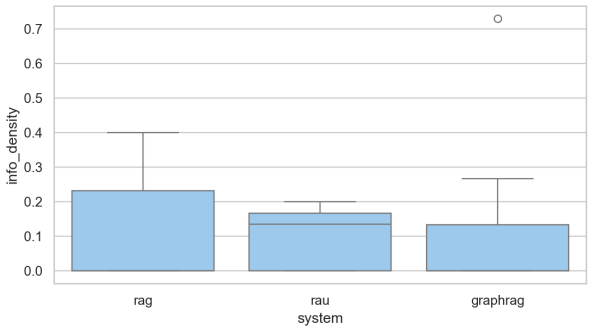
Qualitative inspection focuses on failure modes that are hard to detect from scalar scores alone:



(a) Latency



(b) Faithfulness



(c) Information density

Figure 3: Generation-stage metric distributions over the 7-query subset.

1. **Context overflow**: GraphRAG prompts can exceed local model context budgets, requiring strict prompt budgeting (node count, node length, edge count).
2. **Prompt echoing**: some models echo the provided graph context rather than performing synthesis; we detect this and fall back to a compact graph summary.
3. **Graph shortcutting**: excessive semantic edges can create spurious short paths that connect unrelated clauses, inflating apparent coherence.

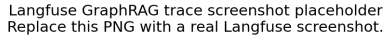
Figures 4 are placeholders for screenshots to be supplied from `doc/paper/images/`:

7 Discussion

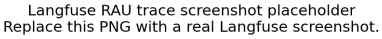
7.1 What the graph adds (and what it does not)

The graph introduces an explicit inductive bias: evidence is no longer treated as an unordered set of chunks but as a structured neighborhood with weighted edges. This facilitates multi-hop retrieval because traversal can surface nodes that are not top-k by similarity alone, but become relevant via strong relational ties.

However, this also introduces new degrees of freedom: threshold τ, traversal depth D, pruning θ, and the weighting hyperparameters (α, β). Each can induce failure modes such as hub-node overload, semantic "short-circuiting" through spurious similarity edges, or over-pruning.



(a) GraphRAG trace



(b) RAU trace

Figure 4: Langfuse trace screenshots (to be provided).

7.2 Trends in GraphRAG node addition

Modern GraphRAG deployments often require continuous ingestion. Common strategies include:

1. **Incremental insertion**: embed new chunks and connect via local heuristics (similarity, metadata, entity links).
2. **Periodic re-indexing**: rebuild embeddings and graph structure in batches.
3. **Hybrid approaches**: insert incrementally but periodically re-optimize communities/edges.

Surveys emphasize that graph construction is configurable and domain-dependent, spanning knowledge graphs, text-chunk graphs, and hybrid graphs [3, 9].

7.3 Update regimes: incremental insertion vs rebuild

We operationalize "graph integrity" using computable network diagnostics. Starting from an initial graph built on 80% of chunks, we compare:

1. **Incremental insertion**: add the remaining 20% of nodes, connecting each node by local similarity and sequential metadata.
2. **Rebuild full**: reconstruct the full graph from scratch on all nodes.

Table 3 reports average degree, clustering coefficient, largest connected component fraction (LCC), and edge-set Jaccard similarity to the full rebuild baseline.

Table 3: Incremental insertion vs rebuild (graph-level diagnostics).

Regime	Avg degree	Clustering	LCC fraction	Edge Jaccard
initial 80pct	4.022	0.228	0.723	0.653
incremental insert	4.842	0.195	0.917	0.983
rebuild full	4.926	0.197	0.917	1.000

7.4 A complexity-science argument via topology and information integration

The intuition behind "information integration" is that useful reasoning arises not from isolated facts but from their structured interdependence. IIT formalizes this via measures of irreducibility and integrated information Φ [8, 11]. Yet exact evaluation becomes intractable for large systems due to the number of partitions; computable relaxations have been proposed [10]. Estimation methods based on algorithmic complexity and dynamic querying illustrate possible approximations, often focusing on restricted (e.g., binary) settings [4].

For GraphRAG, the engineering-relevant question is not whether a graph is "integrated" in the IIT sense, but whether incremental updates preserve the navigation properties that make graph-guided retrieval effective: coherent neighborhoods, stable communities, and interpretable paths. From complex network theory, node insertion can shift degree distributions, clustering coefficients, and path-length properties in ways that change global navigability [1, 7, 12]. From a topology-inspired view, adding edges changes the clique complex induced by a similarity graph, potentially altering connectivity and higher-order structure; this matters because multi-hop retrieval often exploits the existence of short paths through semantically dense regions. These considerations support a conservative systems policy: when update volume is non-trivial, periodic rebuild or re-optimization can be more reliable than purely local insertion, unless stronger constraints or calibration are enforced.

For GraphRAG, the central question becomes: does incremental node insertion preserve (or improve) the graph's integrative structure, or does it accumulate local edges that degrade global coherence? Complex network theory suggests that adding nodes can shift degree distributions, clustering coefficients, and path length properties in ways that change global navigability [1, 7, 12]. From this perspective, periodic re-construction (or re-optimization) may be more reliable than purely local updates when global properties matter for retrieval and reasoning. This is not a final conclusion, but a hypothesis that can be tested using a combination of retrieval diagnostics (coverage/coherence) and graph-theoretic measurements.

8 Limitations and future work

This baseline intentionally excludes higher-order graph learning (e.g., GNN encoders) and focuses on explicit traversal. Future work should evaluate: (i) labeled relevance sets, (ii) ablations over $\tau, D, \theta, \alpha, \beta$, (iii) dynamic graph update regimes, and (iv) principled integration proxies that remain computable at scale.

9 Conclusion

We presented a minimal, explicit GraphRAG baseline whose behavior can be inspected end-to-end: chunk graph construction, weighted traversal, subgraph extraction, and relationship-aware generation. Empirical results show stable retrieval-stage diagnostics over a batch of contract queries. Finally, we argued that "adding nodes" is not a neutral operation: it may alter a graph's integrative structure, motivating a complexity-science lens (including IIT-inspired considerations) for evaluating update strategies.

A Benchmark queries

The benchmark query list is defined in the evaluation script used to generate `results_metrics.csv`. For convenience, the 25 queries are reproduced here:

1. Identify risks involving both termination and liability clauses
2. What obligations survive termination and how do they interact with limitation of liability
3. Find conflicts between indemnification and limitation of liability
4. Identify risks where confidentiality obligations collide with disclosure or audit rights
5. Assess whether termination triggers payment obligations or refund obligations
6. Summarize termination for convenience and termination for cause
7. Summarize limitation of liability and excluded damages
8. Summarize indemnity and defense obligations
9. Summarize confidentiality scope, exclusions, and duration
10. Summarize governing law and venue
11. Summarize assignment restrictions and change of control
12. Summarize force majeure conditions and notice requirements
13. Summarize warranty disclaimers and remedy limitations
14. Summarize payment terms and late payment penalties
15. Summarize dispute resolution, arbitration, and escalation steps
16. Summarize IP ownership and license grants
17. Summarize audit rights and record retention requirements
18. Summarize non-solicitation and non-compete restrictions
19. Summarize data security obligations and breach notification requirements
20. Summarize insurance requirements and proof of coverage
21. Identify clauses that could create unlimited liability exposure
22. Identify clauses that could allow unilateral changes to pricing or scope
23. Identify clauses that shift regulatory compliance risk to one party
24. Identify clauses that create unclear acceptance criteria or deliverables
25. Identify clauses that create broad termination rights with short notice

References

[1] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, 1999. doi: 10.1126/science.286.5439.509. URL https://doi.org/10.1126/science.286.5439.509.

[2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. doi: 10.48550/arXiv.2404.16130. URL https://arxiv.org/abs/2404.16130.

[3] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, Xianfeng Tang, Qi He, Zhigang Hua, Bo Long, Tong Zhao, Neil Shah, Amin Javari, Yinglong Xia, and Jiliang Tang. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309*, 2025. doi: 10.48550/arXiv.2501.00309. URL https://arxiv.org/abs/2501.00309.

[4] Alberto Hernández-Espinosa, Narsis A. Kiani, and Hector Zenil. Estimations of integrated information based on algorithmic complexity and dynamic querying. In *Complexity and Data Analysis*. World Scientific, 2021. doi: 10.1142/9789811235726_0005. URL https://www.worldscientific.com/doi/10.1142/9789811235726_0005.

[5] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020. URL https://arxiv.org/abs/2004.04906.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv preprint arXiv:2005.11401*, 2020. URL https://arxiv.org/abs/2005.11401.

[7] Mark Newman. *Networks: An Introduction*. Oxford University Press, 2010.

[8] Masafumi Oizumi, Larissa Albantakis, and Giulio Tononi. From the phenomenology to the mechanisms of consciousness: Integrated information theory 3.0. *PLoS Computational Biology*, 10(5):e1003588, 2014. doi: 10.1371/journal.pcbi.1003588. URL https://doi.org/10.1371/journal.pcbi.1003588.

[9] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921*, 2024. URL https://arxiv.org/abs/2408.08921.

[10] Max Tegmark. Improved measures of integrated information. *PLoS Computational Biology*, 12(11):e1005123, 2016. doi: 10.1371/journal.pcbi.1005123. URL https://doi.org/10.1371/journal.pcbi.1005123.

[11] Giulio Tononi. An information integration theory of consciousness. *BMC Neuroscience*, 5:42, 2004. doi: 10.1186/1471-2202-5-42. URL https://doi.org/10.1186/1471-2202-5-42.

[12] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of 'small-world' networks. *Nature*, 393:440–442, 1998. doi: 10.1038/30918. URL https://doi.org/10.1038/30918.